Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

TEAM DETAILS

S.NO	NAME	COLLEGE	CONTACT
1	SABARIYA S	St.joseph's college for women,Tiruppur	ssabariya17@gmail.com
2	DHAKSHITHA S P	St.joseph's college for women,Tiruppur	dhakshukrush@gmail.com
3	SHERIN FATHIMA F	St.joseph's college for women,Tiruppur	sherinfathima2006@gmail.com
4	THAFHEEMA M	St.joseph's college for women,Tiruppur	thafheemam@gmail.com

PROJECT DETAILS

Title of the project:

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

- **Zero Trust Network Access (ZTNA).**

Zero Trust Network Access (ZTNA) is a modern security approach that ensures secure access to applications by verifying every user, device, and connection before granting permission. Traditional network security models assume that users inside the network can be trusted, but the Zero Trust model follows the principle of **"never trust, always verify."** In this project, the ZTNA architecture is implemented using **Zscaler**, a cloud-based security platform that enables secure user-to-application connectivity without exposing internal networks to the internet. The system allows users to securely access private applications from any location while ensuring strict identity verification and access control. Instead of providing full network access like traditional VPNs, ZTNA grants **application-level access** based on policies such as user identity, device posture, and location. Zscaler acts as a secure broker between users and applications, ensuring that applications remain hidden from unauthorized users and reducing the attack surface.

This project demonstrates how **Zscaler's Zero Trust architecture** protects enterprise resources by continuously authenticating users, inspecting traffic, and enforcing security policies. By implementing ZTNA, organizations can improve security, reduce the risk of cyberattacks, and provide seamless remote access to applications without compromising network safety. The solution is especially useful in modern cloud environments where users work remotely and access applications from different devices and locations.

Abstract:

With the rapid growth of cloud computing and remote work, traditional network security models such as VPNs are no longer sufficient to protect organizational resources. Zero Trust Network Access (ZTNA) is a modern security framework that follows the principle of **"never trust, always verify,"** ensuring that every user and device is authenticated and authorized before accessing applications. This project focuses on implementing ZTNA using **Zscaler**, a cloud-based security platform that provides secure and controlled user-to-application connectivity. The system ensures that users can securely access internal applications without exposing the entire network to potential threats. Instead of granting broad network access, ZTNA provides **application-level access** based on identity, device posture, and security policies. Zscaler acts as an intermediary that authenticates users, verifies device security, and enforces access policies while keeping applications hidden from the public internet.The proposed solution enhances security by reducing the attack surface, preventing unauthorized access, and protecting sensitive organizational data. It also improves user experience by enabling seamless and secure remote access to applications from any location. By adopting ZTNA with Zscaler, organizations can strengthen their cybersecurity posture, support modern work environments, and ensure safe connectivity between users and applications.

Use Case Scenarios:

Scenario 1: Remote Employee Access to Internal Application.

In modern organizations, many employees work remotely and need to access internal applications from outside the company network. Traditional VPN-based systems often provide full network access, which increases security risks. With Zero Trust Network Access (ZTNA), the system verifies the identity of the employee before allowing access. The user attempts to log in to the organization's internal application using their credentials. The ZTNA system checks user identity, device posture, and security policies before granting permission. Instead of giving access to the entire network, the system allows access only to the specific application requested. This approach ensures that sensitive company resources remain protected while still enabling employees to work efficiently from remote locations.

Output:

- The user identity is verified through secure authentication.
- The device security posture is checked before granting access.
- A secure connection is established between the user and the application.
- Access is granted only to the specific authorized applications.

Scenario 2: Unauthorized User Attempting Access.

Cyber attackers often try to gain unauthorized access to internal enterprise applications by using stolen credentials or attempting to bypass security systems. In this scenario, an unknown user tries to access a private company application without valid authentication. The Zero Trust model ensures that no user or device is trusted by default. When the access request is made, the ZTNA system immediately verifies the identity of the user and checks the access policy. Since the user does not meet the authentication requirements, the system blocks the connection attempt. The application itself remains invisible to unauthorized users, preventing attackers from even detecting its presence. All suspicious activities are recorded for monitoring and analysis.

Output:

- Invalid authentication is detected by the system.
- The access request is automatically denied.
- The application remains hidden from unauthorized users.
- The access attempt is logged for security monitoring.

Scenario 3: Device Security Compliance Check.

Organizations often enforce security policies to ensure that only secure and trusted devices can access company resources. In this scenario, a user attempts to access an internal application using a device that may not meet the organization's security requirements. The ZTNA system performs a device posture check, which includes verifying antivirus status, operating system updates, and device compliance. If the device fails to meet the required security standards, the system restricts or blocks access. This helps prevent potential threats from compromised or vulnerable devices. By ensuring that only secure devices can connect to applications, the organization can reduce the risk of malware attacks and data breaches.

Output:

- The device security status is analyzed by the system.
- Non-compliant devices are automatically detected.
- Access to the application is restricted or denied.
- The user receives instructions to update device security.

Scenario 4: Role-Based Access Control.

In an organization, different employees have different responsibilities and access requirements. For example, managers, developers, and interns may need access to different applications. In this scenario, the ZTNA system enforces role-based access control policies. Based on predefined security policies, access is granted only to the applications required for that role. This ensures that employees cannot access sensitive data or systems that are not related to their job responsibilities. By implementing role-based access control, organizations can improve security while maintaining efficient workflow.

Output:

- The user's identity and role are verified.
- Access policies determine the allowed applications.
- Users receive access only to authorized resources.
- Sensitive applications remain restricted to authorized roles.

Scenario 5: Secure Access from Public Network.

Employees often access company applications while traveling or working from public locations such as cafes, airports, or hotels. Public Wi-Fi networks are not always secure and may expose users to cyber threats. In this scenario, a user attempts to access a company application through a public network. This ensures that sensitive company data is protected even when accessed from untrusted networks. The user experiences secure and seamless access without exposing the internal network infrastructure.

Output:

- The connection is routed through the secure ZTNA cloud gateway.
- Data traffic is encrypted and monitored for threats.
- The user securely connects to the required application.
- Risks from public networks are minimized through security policies.

STAGE – 1

Title of the project:

Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Overview:

Zero Trust Network Access focuses on providing **secure user-to-application connectivity** rather than traditional network-level access. Instead of allowing users to connect to the entire network, ZTNA grants access only to specific applications based on predefined security policies. These policies consider multiple factors such as user identity, device security posture, location, time of access, and compliance with organizational security standards. This ensures that only authorized users with trusted devices can access critical applications, while unauthorized users are blocked automatically.

In this project, ZTNA is implemented using **Zscaler**, a leading cloud-based cybersecurity platform that delivers secure access to applications and services without exposing internal networks to the internet. Zscaler acts as a secure cloud gateway that connects users directly to applications through an encrypted and monitored connection. When a user attempts to access an application, the system verifies their identity through authentication mechanisms and evaluates the security posture of the device being used. Only after successful verification does the system allow access to the requested application.

One of the key advantages of using Zscaler for Zero Trust Network Access is that it eliminates the need for traditional VPN infrastructure. VPNs often create security risks because they provide users with full network access after authentication. In contrast, ZTNA limits access strictly to authorized applications, preventing attackers from moving laterally within the network even if a device becomes compromised. Additionally, applications remain hidden from the public internet, reducing the risk of direct attacks or unauthorized discovery.

Another important feature of ZTNA with Zscaler is continuous monitoring and traffic inspection. All user connections and data traffic are analyzed in real time to detect potential threats, malware, or suspicious activities. Security policies are dynamically enforced to ensure that only safe and compliant connections are allowed. This proactive approach helps organizations identify and prevent security incidents before they cause significant damage.

The system also improves user experience by enabling seamless and secure access to applications from any location. Employees working from home, traveling, or using public networks can safely connect to enterprise applications without compromising security. By verifying both the user and the device, the system ensures that access remains secure regardless of where the user is located.

Furthermore, the Zero Trust approach supports modern cloud-based environments where applications may be hosted in data centers, private clouds, or public cloud platforms. Zscaler integrates with these environments to provide a unified security framework that protects applications across multiple infrastructures. This makes it easier for organizations to manage security policies, monitor user activity, and maintain compliance with security standards.

Overall, the implementation of **Zero Trust Network Access with Zscaler** provides a strong and scalable security solution for modern enterprises. It reduces the risk of cyberattacks, protects sensitive data, and ensures secure access to applications without exposing the internal network. By adopting this model, organizations can enhance their cybersecurity posture while supporting flexible work environments and digital transformation. This project demonstrates how ZTNA technology can be effectively used to secure user-to-application connectivity, improve access control, and strengthen the overall security architecture of an organization.

List of Vulnerability Table:-

S.no	Category Name	Vulnerability Name	CWE/CVE - No
1	Broken Access Control	Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')	CWE-22
2	Security Misconfiguration	J2EE Misconfiguration: Data Transmission Without Encryption	CWE-5
3	Software Supply Chain Failures	Unimplemented or Unsupported Feature in UI	CWE-447
4	Cryptographic Failures	Weak Encoding For Password	CWE-261
5	Injection	Improper Input Validation	CWE-20
6	Insecure Design	External control of file name or path	CWE-73

7	Authentication Failures	Improper Authentication	CWE-287
8	Software or Data integrity Failures	Untrusted Search Path	CWE-426
9	Security Logging and Alerting Failures	Insufficient Logging	CWE-778
10	Mishandling of Exceptional Conditions	Unchecked Return Value	CWE-252

REPORTS

1)Vulnerability Name: Improper Limitation of a Pathname to a Restricted Directory ('Path Traversal')

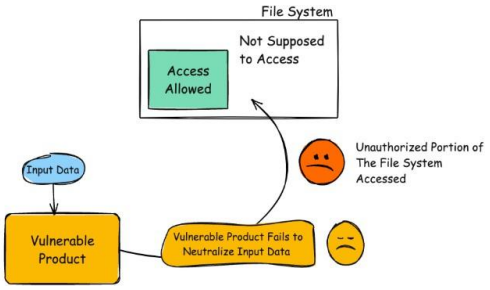
CWE/CVE: CWE-22

OWASP/SANS Category: A01:2025 Broken Access Control

Description: Path Traversal is a vulnerability that occurs when an application does not properly validate file path inputs from users. Attackers can manipulate directory paths (e.g.,.. /) to access files and directories outside the intended restricted folder.

Business Impact: Attackers may gain unauthorized access to sensitive files such as configuration files, passwords, or system data. This can lead to data breaches, exposure of confidential information, and potential system compromise. Such incidents may result in financial losses, reputational damage, and legal or regulatory consequences for the organization.

The product uses external input to construct a pathname that is intended to identify a file or directory that is located underneath a restricted parent directory, but the product does not properly neutralize special elements within the pathname that can cause the pathname to resolve to a location that is outside of the restricted directory.



Recommendations: -

1. **Input Validation and Sanitization:**
   Validate and sanitize all user inputs that are used to access files to ensure that they do not contain directory traversal characters like ../.
2. **Use Secure File Access Methods:**
   Restrict file access to predefined directories and avoid using direct user input to construct file paths.
3. **Implement Access Controls:**
   Ensure proper authentication and authorization mechanisms are in place to prevent unauthorized users from accessing sensitive resources.
4. **Use Security Libraries and Frameworks:**
   Utilize secure coding practices and built-in security features provided by programming frameworks to handle file operations safely.
5. **Regular Security Testing:**
   Conduct vulnerability scanning, penetration testing, and code reviews to detect and fix path traversal vulnerabilities early.

2)Vulnerability Name: J2EE Misconfiguration: Data Transmission Without Encryption.

CWE/CVE : CWE-5

OWASP/SANS Category: A02:2025 Security Misconfiguration

Description: Security Misconfiguration occurs when security settings in applications, servers, or databases are not properly configured or are left with default settings. This can expose systems to attackers who exploit these weaknesses to gain unauthorized access.

Business Impact: Attackers may exploit misconfigured systems to access sensitive data or system resources. It can lead to data breaches, service disruption, or unauthorized system control. This may cause financial loss, reputational damage, and potential legal consequences for the organization.

Recommendations: -

1. **Remove Default Configurations:**
   Change default usernames, passwords, and unnecessary settings to secure configurations.
2. **Regular Security Updates:**
   Keep servers, applications, and software updated with the latest security patches.
3. **Disable Unnecessary Services:**
   Turn off unused features, services, and ports to reduce the attack surface.
4. **Implement Proper Access Control:**
   Ensure only authorized users have access to sensitive system components.
5. **Perform Regular Security Audits:**
   Conduct vulnerability scans and configuration reviews to identify and fix misconfigurations.

3)Vulnerability Name: Unimplemented or Unsupported Feature in UI

CWE/CVE: CWE-447

OWASP/SANS Category: A03:2025 Software Supply Chain Failures

Description: This vulnerability occurs when a user interface includes features or options that are not fully implemented or supported by the system. Attackers may exploit these unfinished or inactive components to bypass controls or trigger unexpected system behavior.

Business Impact: Unimplemented features may expose hidden system functions or sensitive information. Attackers could exploit these weaknesses to bypass security controls or cause system errors. This can lead to service disruption, security risks, and reduced user trust in the application.

Recommendations: -

1. **Remove Incomplete Features:**
   All unfinished or unsupported features should be removed from the user interface until they are fully developed and tested.
2. **Disable Unused UI Components:**
   Buttons, links, or menu options that are not functional should be disabled to prevent users or attackers from interacting with them.
3. **Implement Proper Error Handling:**
   The system should display generic error messages and avoid revealing internal system information or code details.
4. **Conduct Regular Security Testing:**
   Perform regular vulnerability scans, penetration testing, and code reviews to identify hidden or unsupported features.
5. **Follow Secure Development Practices:**
   Developers should follow secure coding standards and thoroughly test new features before deploying them to production.

4) Vulnerability Name: Weak Encoding For Password

CWE/CVE: CWE-261

OWASP/SANS Category: A04:2025 Cryptographic Failures

Description: Weak encoding for passwords occurs when passwords are stored using insecure or easily reversible encoding methods instead of strong hashing algorithms. This makes it easier for attackers to decode or retrieve the original passwords if the database is compromised.

Business Impact: Attackers may be able to decode stored passwords and gain unauthorized access to user accounts. Compromised accounts can lead to data breaches, identity theft, and misuse of sensitive information. This can damage the organization's reputation and lead to financial or legal consequences.

Vulnerability Mapping Notes	
Usage	**ALLOWED** *(this CWE ID may be used to map to real-world vulnerabilities)*
Reason	Acceptable-Use
Rationale	This CWE entry is at the Base level of abstraction, which is a preferred level of abstraction for mapping to the root causes of vulnerabilities.
Comments	Carefully read both the name and description to ensure that this mapping is an appropriate fit. Do not try to 'force' a mapping to a lower-level Base/Variant simply to comply with this preferred level of abstraction.

Example 1

The following code reads a password from a properties file and uses the password to connect to a database.

Example Language: **Java** *(bad code)*

```
...
Properties prop = new Properties();
prop.load(new FileInputStream("config.properties"));
String password = Base64.decode(prop.getProperty("password"));
DriverManager.getConnection(url, usr, password);
...
```

This code will run successfully, but anyone with access to config.properties can read the value of password and easily determine that the value has been base 64 encoded. If a devious employee has access to this information, they can use it to break into the system.

Recommendations: -

1. **Use Strong Hashing Algorithms:**
   Passwords should be stored using strong hashing algorithms such as **bcrypt, Argon2, or PBKDF2** instead of simple encoding.
2. **Implement Password Salting:**
   Add a unique salt value to each password before hashing to protect against rainbow Table attacks.

3. **Enforce Strong Password Policies:**
   Require users to create complex passwords with a mix of letters, numbers, and special Characters.
4. **Protect the Authentication System:**
   Use secure authentication mechanisms and restrict access to password storage Databases.
5. **Perform Regular Security Audits:**
   Regularly review password storage mechanisms and perform vulnerability assessments to detect weak encoding methods.

5)Vulnerability Name: Improper Input Validation.

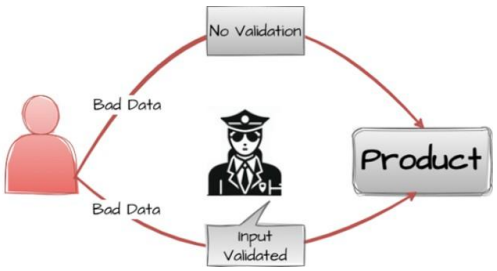
CWE/CVE: CWE-20

OWASP/SANS Category: A05:2025 Injection

Description: Injection vulnerabilities occur when untrusted user input is sent to an interpreter such as a database or command shell without proper validation. Attackers can insert malicious code (e.g., SQL, OS commands) to manipulate the system or access unauthorized data.

Business Impact: Attackers may gain unauthorized access to sensitive data stored in the database. It can lead to data modification, deletion, or complete compromise of the application. This may cause financial loss, reputational damage, and legal or regulatory issues.

The product receives input or data, but it does not validate or incorrectly validates that the input has the properties that are required to process the data safely and correctly.



Vulnerability Mapping Notes	
Usage	**DISCOURAGED** *(this CWE ID should not be used to map to real-world vulnerabilities)*
Reasons	Frequent Misuse, Frequent Misinterpretation, Abstraction
Rationale	CWE-20 is commonly misused in low-information vulnerability reports when lower-level CWEs could be used instead, or when more details about the vulnerability are available [REF-1287]. It is not useful for trend analysis. It is also a level-1 Class (i.e., a child of a Pillar). Finally, it is often used when the root cause issue is related to how input is incorrectly transformed, instead of "validated" to be correct as-is.
Comments	Within CWE, the "input validation" term focuses on the act of checking whether an input is already safe, which is different from other techniques that ensure safe processing of input. Carefully perform root-cause analysis to be sure that the issue is not due to techniques that attempt to transform potentially-dangerous input into something safe, such as filtering (CWE-790) - which attempts to remove dangerous inputs - or encoding/escaping (CWE-116), which attempts to ensure that the input is not misinterpreted when it is included in output to another component. If the issue is truly due to imroper input validation, consider using lower-level children such as Improper Use of Validation Framework (CWE-1173) or improper validation involving specific types or properties of input such as Specified Quantity (CWE-1284); Specified Index, Position, or Offset (CWE-1285); Syntactic Correctness (CWE-1286); Specified Type (CWE-1287); Consistency within Input (CWE-1288); or Unsafe Equivalence (CWE-1289).
Suggestions	

CWE-ID	Comment
CWE-1284	Specified Quantity
CWE-1285	Specified Index, Position, or Offset
CWE-1286	Syntactic Correctness
CWE-1287	Specified Type
CWE-1288	Consistency within Input
CWE-1289	Unsafe Equivalence
CWE-116	Improper Encoding or Escaping of Output
CWE-790	Improper Filtering of Special Elements

Recommendations:

1. **Input Validation:**
   Validate and sanitize all user inputs to ensure they do not contain malicious code or Commands.
2. **Use Parameterized Queries:**
   Use prepared statements or parameterized queries to prevent attackers from injecting SQL commands.
3. **Implement Proper Access Controls:**
   Restrict database permissions so that users and applications can only access necessary Data.
4. **Use Security Frameworks:**
   Use secure development frameworks and libraries that automatically protect against injection attacks.
5. **Perform Regular Security Testing:**
   Conduct regular vulnerability scans, penetration testing, and code reviews to detect and fix injection vulnerabilities.

6)Vulnerability Name: External control of file name or path.

CWE/CVE: CWE-73

OWASP/SANS Category: A06:2025 Insecure Design

Description: This vulnerability occurs when an application allows users to control or modify the file name or file path used by the system without proper validation. Attackers can manipulate the file path to access, modify, or overwrite sensitive files on the server.

Business Impact: Attackers may gain unauthorized access to sensitive files stored on the system.It can lead to data leakage, file modification, or system compromise. This may result in security breaches, financial losses, and damage to the organization's reputation.

Extended Description

This could allow an attacker to access or modify system files or other files that are critical to the application.

Path manipulation errors occur when the following two conditions are met:

1. An attacker can specify a path used in an operation on the filesystem.
2. By specifying the resource, the attacker gains a capability that would not otherwise be permitted.

For example, the program may give the attacker the ability to overwrite the specified file or run with a configuration controlled by the attacker.

Common Consequences

Impact	Details
Read Files or Directories; Modify Files or Directories	**Scope: Integrity, Confidentiality** The application can operate on unexpected files. Confidentiality is violated when the targeted filename is not directly readable by the attacker.
Modify Files or Directories; Execute Unauthorized Code or Commands	**Scope: Integrity, Confidentiality, Availability** The application can operate on unexpected files. This may violate integrity if the filename is written to, or if the filename is for a program or other form of executable code.
DoS: Crash, Exit, or Restart; DoS: Resource Consumption (Other)	**Scope: Availability** The application can operate on unexpected files. Availability can be violated if the attacker specifies an unexpected file that the application modifies. Availability can also be affected if the attacker specifies a filename for a large file, or points to a special device or a file that does not have the format that the application expects.

Recommendations: -

1. **Validate User Input:**
   Validate and sanitize all user inputs that are used to create or access file names and Paths.
2. **Restrict File Access:**
   Allow file operations only within specific authorized directories to prevent unauthorized access.
3. **Use Secure File Handling:**
   Avoid directly using user input in file system commands and use secure APIs for file operations.
4. **Implement Access Control:**
   Ensure that only authorized users have permission to access or modify important file
5. **Perform Security Testing:**
   Regularly conduct vulnerability scanning and code reviews to detect file path manipulation issues.

7) Vulnerability Name: Improper Authentication.

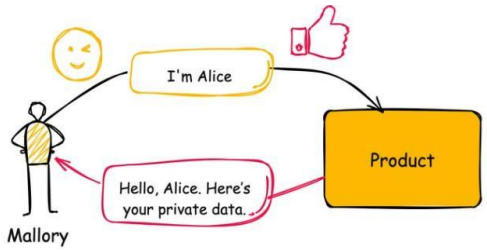
CWE/CVE CWE-287

OWASP/SANS Category: A07:2025 Authentication Failure

Description: Improper Authentication occurs when an application fails to correctly verify the identity of users before granting access to resources. Attackers can exploit weak authentication mechanisms to bypass login controls and gain unauthorized access.

Business Impact: Unauthorized users may gain access to sensitive systems or user accounts. This can lead to data theft, account takeover, or misuse of confidential information. Such incidents may cause financial loss, reputational damage, and legal consequences for the organization.

When an actor claims to have a given identity, the product does not prove or insufficiently proves that the claim is correct.



Alternate Terms	
authentification	An alternate term is "authentification", which appears to be most commonly used by people from non-English-speaking countries.
AuthN	"AuthN" is typically used as an abbreviation of "authentication" within the web application security community. It is also distinct from "AuthZ," which is an abbreviation of "authorization." The use of "Auth" as an abbreviation is discouraged, since it could be used for either authentication or authorization.
AuthC	"AuthC" is used as an abbreviation of "authentication," but it appears to used less frequently than "AuthN."

Recommendations: -

1. **Implement Strong Authentication:**
   Use secure authentication methods such as multi-factor authentication (MFA) to verify user identity.
2. **Enforce Strong Password Policies:**
   Require complex passwords and regular password updates to reduce the risk of unauthorized access.
3. **Secure Session Management:**
   Use secure session tokens and ensure sessions expire after a certain period of Inactivity.

4. **Limit Login Attempts:**
   Implement account lockout mechanisms after multiple failed login attempts to prevent brute-force attacks.
5. **Regular Security Monitoring:**
   Monitor authentication activities and perform regular security testing to detect and prevent authentication vulnerabilities.

8)Vulnerability Name: Untrusted Search Path

CWE/CVE: CWE-426

OWASP/SANS Category: A08:2025 Software or Data Integrity Failures

Description: Untrusted Search Path vulnerability occurs when an application searches for executable files or libraries in directories that may be controlled by attackers. This allows attackers to place malicious files in those locations so the system executes them instead of the legitimate ones.

Business Impact: Attackers may execute malicious programs with the same privileges as the application. This can lead to system compromise, unauthorized access, or malware installation. It may result in data loss, operational disruption, and damage to the organization's reputation.

Extended Description

This might allow attackers to execute their own programs, access unauthorized data files, or modify configuration in unexpected ways. If the product uses a search path to locate critical resources such as programs, then an attacker could modify that search path to point to a malicious program, which the targeted product would then execute. The problem extends to any type of critical resource that the product trusts.

Some of the most common variants of untrusted search path are:

- In various UNIX and Linux-based systems, the PATH environment variable may be consulted to locate executable programs, and LD_PRELOAD may be used to locate a separate library.
- In various Microsoft-based systems, the PATH environment variable is consulted to locate a DLL, if the DLL is not found in other paths that appear earlier in the search order.

Recommendations: -

1. **Use Absolute File Paths:**
   Always specify full and trusted paths for executables and system files instead of relying on search paths.
2. **Restrict Directory Permissions:**
   Limit write permissions to system directories to prevent attackers from placing malicious files.
3. **Validate Executable Sources:**
   Ensure that applications only load executables or libraries from trusted and verified locations.
4. **Implement Secure Configuration:**
   Configure the system environment variables and search paths securely to avoid untrusted directories.
5. **Perform Security Audits:**
   Regularly review application configurations and conduct vulnerability scans to detect untrusted search path issues.

9)Vulnerability Name: Insufficient Logging

CWE/CVE: CWE-778

OWASP/SANS Category: A09:2025 Security logging And Alerting Failures.

Description: Insufficient Logging occurs when an application or system does not properly record security-related events or user activities. Without proper logs and monitoring, suspicious or malicious activities may go undetected.

Business Impact: Security incidents may not be detected quickly, allowing attackers to continue their activities unnoticed. It becomes difficult to investigate attacks or identify the root cause of security breaches. This can lead to data loss, financial damage, and serious harm to the organization's reputation.

Extended Description

When security-critical events are not logged properly, such as a failed login attempt, this can make malicious behavior more difficult to detect and may hinder forensic analysis after an attack succeeds.

As organizations adopt cloud storage resources, these technologies often require configuration changes to enable detailed logging information, since detailed logging can incur additional costs. This could lead to telemetry gaps in critical audit logs. For example, in Azure, the default value for logging is disabled.

Vulnerability Mapping Notes	
Usage	ALLOWED *(this CWE ID may be used to map to real-world vulnerabilities)*
Reason	Acceptable-Use
Rationale	This CWE entry is at the Base level of abstraction, which is a preferred level of abstraction for mapping to the root causes of vulnerabilities.
Comments	Carefully read both the name and description to ensure that this mapping is an appropriate fit. Do not try to 'force' a mapping to a lower-level Base/Variant simply to comply with this preferred level of abstraction.

Recommendations: -

1. **Implement Comprehensive Logging:**
   Record important events such as login attempts, access to sensitive data, and system changes.
2. **Enable Real-Time Monitoring:**
   Use monitoring tools to detect unusual activities and potential security threats in real time.
3. **Protect Log Integrity**:
   Store logs securely and prevent unauthorized modification or deletion.
4. **Regular Log Review:**
   Periodically review and analyze logs to identify suspicious behavior.
5. **Use Security Information and Event Management (SIEM):**
   Implement SIEM tools to collect, analyze, and respond to security events efficiently.

10) Vulnerability Name: Unchecked Return Value

CWE/CVE: CWE-252

OWASP/SANS Category: A09:2025 Security logging And Alerting Failures.

Description: Unchecked Return Value occurs when a program fails to verify the return value of a function or system call. This may cause the application to continue execution even when an error or failure has occurred.

Business Impact: Ignoring return values can lead to unexpected system behavior or application crashes. Attackers may exploit these weaknesses to bypass security checks or trigger system failures. This can result in service disruption, data corruption, or reduced reliability of the application.

Description

The product does not check the return value from a method or function, which can prevent it from detecting unexpected states and conditions.

Extended Description

Two common programmer assumptions are "this function call can never fail" and "it doesn't matter if this function call fails". If an attacker can force the function to fail or otherwise return a value that is not expected, then the subsequent program logic could lead to a vulnerability, because the product is not in a state that the programmer assumes. For example, if the program calls a function to drop privileges but does not check the return code to ensure that privileges were successfully dropped, then the program will continue to operate with the higher privileges.

In the following example, it is possible to request that memcpy move a much larger segment of memory than assumed:

Example Language: **C** *(bad code)*

```c
int returnChunkSize(void *) {

  /* if chunk info is valid, return the size of usable memory,

  * else, return -1 to indicate an error

  */
  ...
}
int main() {
  ...
  memcpy(destBuf, srcBuf, (returnChunkSize(destBuf)-1));
  ...
}
```

If returnChunkSize() happens to encounter an error it will return -1. Notice that the return value is not checked before the memcpy operation (CWE-252), so -1 can be passed as the size argument to memcpy() (CWE-805). Because memcpy() assumes that the value is unsigned, it will be interpreted as MAXINT-1 (CWE-195), and therefore will copy far more memory than is likely available to the destination buffer (CWE-787, CWE-788).

Recommendations:-

1. **Validate Return Values:**
   Always check the return value of functions and system calls to ensure operations are completed successfully.
2. **Implement Proper Error Handling:**
   Handle errors appropriately by logging them and taking corrective actions when failures Occurs.
3. **Use Secure Coding Practices:**
   Follow secure coding standards that require validation of function outputs.
4. **Conduct Code Reviews:**
   Perform regular code reviews to ensure developers are properly checking return values.
5. **Perform Testing and Debugging:**
   Use static analysis tools and thorough testing to detect unchecked return value issues Early.

STAGE – 2

Overview :-

Nessus is a vulnerability scanner developed by Tenable, Inc. It is a remote scanner, meaning that it does not need to be installed on the target host to scan it. Nessus works by sending a variety of probes to the target host in order to identify vulnerabilities. These probes can include network traffic, operating system commands, and application requests. Nessus uses a plugin-based architecture. Plugins are small pieces of code that are designed to identify specific vulnerabilities. There are over 1200 plugins available for Nessus, covering a wide range of vulnerabilities in operating systems, applications, and network devices. When Nessus scans a host, it first uses a discovery plugin to identify the operating system and services running on the host. Once the host has been discovered, Nessus runs a series of plugins to test for vulnerabilities. For each vulnerability that Nessus finds, it provides a detailed report that includes the severity of the vulnerability, the steps to reproduce the vulnerability, and the steps to remediate the vulnerability.

Nessus can be used to scan a variety of targets, including single hosts, networks, and web applications. It can also be used to scan specific ports and services. Nessus can be configured to run scans on a schedule or manually.

Here is a step-by-step explanation of how Nessus works:

1. Nessus connects to the target host using a variety of protocols, including TCP, UDP, and ICMP.
2. Nessus sends a series of probes to the target host in order to identify its operating system and services.
3. Nessus runs a series of plugins to test for vulnerabilities.
4. Nessus generates a report that includes the severity of each vulnerability, the steps to reproduce the vulnerability, and the steps to remediate the vulnerability.

Nessus is a valuable tool for security professionals who need to identify and remediate vulnerabilities in their networks. It is easy to use and provides comprehensive reports that can be used to priorities remediation efforts.

Here are some of the benefits of using Nessus:

- **Comprehensive vulnerability coverage:**
  Nessus has over 1200 plugins available, covering a wide range of vulnerabilities in operating systems, applications, and network devices.
- **Easy to use:**
  Nessus has a user-friendly interface that makes it easy to configure and run scans

- **Comprehensive reporting:**
  Nessus provides detailed reports that include the severity of each vulnerability, the steps to reproduce the vulnerability, and the steps to remediate the vulnerability.
- **Scalability:**
  Nessus can be used to scan a variety of targets, including single hosts, networks, and web applications.
- **Flexibility:**
  Nessus can be configured to run scans on a schedule or manually.

Nessus is a powerful tool that can be used to improve the security of any network. It is a valuable asset for any security professional.

- **Target website:** https://ginandjuice.shop/
- **Target ip address:** 34.249.203.140

List of Vulnerabilities: -

S.no	Vulnerability Name	Severity	Plugins
1	Service Detection	INFO	22964
2	Nessus SYN scanner	INFO	11219
3	Additional DNS Hostnames	INFO	46180
4	HOST Fully Qualified Domain Name (FQDN) Resolution	INFO	12053
5	Nessus Scan Information	INFO	190506
6	OS Fingerprints Detected	INFO	209654

7	Remote Services Not Using Post-Quantum Ciphers	INFO	277650
8	SSL Root Certification Authority Certificate Information	INFO	94761
9	TLS Version1.2 Protocol Detection	INFO	136318
10	Traceroute Information	INFO	10287

REPORTS

1) Vulnerability Name: Service Detection

Severity: Info

Plugin: 22964

Port: 80/tcp/www, 443/tcp/www

Description:

Nessus was able to identify the remote service by its banner or by looking at the error message it sends when it receives an HTTP request.

Solution:

No solution required.

Business Impact:

Service detection reveals information about services running on the target system. Attackers may use this information to identify active services and plan further reconnaissance or targeted attacks against those services.

2) Vulnerability Name: Nessus SYN Scanner

Severity: Info

Plugin: 11219

Port: 80/tcp, 443/tcp

Description:

This plugin is a SYN 'half-open' port scanner. It shall be reasonably quick even against a firewalled target.

Note that SYN scans are less intrusive than TCP (full connect) scans and broken services, but they might cause problems for less robust firewalls and also leave unnoticed connections on the remote target if the network is loaded.

Solution:

Protect your target with an IP filter.

Business Impact:

Open ports such as 80/tcp and 443/tcp indicate that services are accessible on the remote host. If these ports are exposed without proper filtering or security controls, attackers may use them to identify running services and attempt further attacks against those services. This could increase the attack surface of the system and allow unauthorized probing or exploitation attempts.

3) Vulnerability Name: Additional DNS Hostnames

Severity: Info

Plugin: 46180

Port: 443/tcp
Description:
The remote host resolves to additional DNS hostnames. Nessus discovered the following DNS hostnames for the remote host.
Solution:
No solution required.
Business Impact:
The presence of additional DNS hostnames may reveal information about the system or infrastructure. Attackers could use this information to gather details about the network environment and identify potential targets for further attacks.
.

4) Vulnerability Name: OS Fingerprints Detected

Severity: Info

Plugin: 11936

Port: 0

Description:
Nessus was able to detect the remote operating system using various techniques.

Solution:
No solution required.

Business Impact:
Operating system fingerprinting may reveal information about the system running on the remote host. Attackers could use this information to identify the operating system and attempt attacks that specifically target vulnerabilities associated with that operating system.

5) Vulnerability Name: TLS Version 1.2 Protocol Detection

Severity: Info

Plugin: 104743

Port: 443/tcp

Description:
The remote service accepts connections encrypted using TLS 1.2.

Solution:
No solution required.

Business Impact:

The detection of TLS version 1.2 indicates that the remote service supports encrypted communication. While TLS 1.2 is generally considered secure, identifying the protocol version used by a service may provide attackers with information about the security configuration of the system and could assist in further reconnaissance.

6) Vulnerability Name: Traceroute Information

Severity: Info

Plugin: 10287

Port: 0

Description:
It was possible to obtain traceroute information for the remote host.

Solution:
Filter the outgoing ICMP “Time Exceeded” messages (type 11) at the border routers.

Business Impact:
Traceroute information can reveal details about the network path between the scanner and the target host. Attackers could use this information to map the network infrastructure, identify intermediate systems, and gather intelligence about the network layout for planning further attacks.

7) Vulnerability Name: SSL Certificate Information

Severity: Info

Plugin: 10863

Port: 443/tcp

Description:
This plugin gathers information about the SSL certificate on the remote host.

Solution:
No solution required.

Business Impact:
SSL certificate information may reveal details such as the certificate issuer, validity period, and domain names associated with the certificate. Attackers could use this information for reconnaissance to better understand the security configuration of the system and identify potential targets for further attacks.

8) Vulnerability Name: HTTP Server Type and Version

Severity: Info

Plugin: 10107

Port: 80/tcp, 443/tcp

Description:
This plugin attempts to determine the type and version of the remote web server.

Solution:
Configure the web server to hide the server version information.

Business Impact:
Revealing the web server type and version may allow attackers to identify known vulnerabilities associated with that specific server software and version. This information can be used to target the system with exploits that affect that particular server version.

9) Vulnerability Name: DNS Server Cache Snooping Remote Information Disclosure

Severity: Medium

Plugin: 12217

Port: 53/tcp

Description:
The remote DNS server responds to queries for third-party domains that do not have the recursion bit set. This may allow a remote attacker to determine which sites have recently been visited by users of the DNS server.

Solution:
Disable DNS cache snooping on the remote DNS server.

Business Impact:
If DNS cache snooping is possible, attackers may be able to discover domains that users of the DNS server have recently visited. This could lead to disclosure of sensitive information about internal network activity or user behavior, potentially aiding further targeted attacks.

10) Vulnerability Name: SSL Self-Signed Certificate

Severity: Medium

Plugin: 57582

Port: 443/tcp

Description:
The X.509 certificate presented by the remote host is self-signed. This means that the certificate was not issued by a trusted Certificate Authority.

Solution:
Replace the self-signed certificate with a certificate issued by a trusted Certificate Authority.

Business Impact:
Using a self-signed certificate may allow attackers to perform man-in-the-middle attacks because the identity of the server cannot be properly verified by clients. This may expose sensitive data transmitted between the client and the server and reduce the trustworthiness of the secure connection.

.

STAGE – 3

REPORTS:

Tittle :- Enforce least-privilege access through identity-aware, application-specific access policies using Zscaler Private Access (ZPA).

Zero Trust Network Access (ZTNA)

Zero Trust Network Access (ZTNA) is a cybersecurity framework that shifts access control from a network-centric model to an identity-centric model. The foundational principle of ZTNA is "never trust, always verify," meaning that no user or device is trusted by default, regardless of whether they are connecting from inside or outside the corporate network. Unlike traditional remote access solutions that grant users broad access to the entire network, ZTNA creates a secure, application-specific tunnel between the user and the resources they are authorized to access. This ensures that the application remains invisible to unauthorized users and the broader internet, effectively shrinking the attack surface and eliminating the risk of lateral movement across the network.

Zscaler Private Access (ZPA)

Zscaler Private Access (ZPA) is a cloud-delivered ZTNA solution that provides identity- and context- based access to private applications without granting network-level access. ZPA operates on a "service- initiated" model, where users connect to the Zscaler cloud, and the cloud brokers a secure connection to the specific application via ZPA App Connectors deployed in the data center or cloud environment. Because ZPA never places the user on the corporate network, internal IP addresses and network infrastructure are completely hidden. Access policies are dynamically enforced based on user identity, group membership, device posture, and location, ensuring that each user sees only the applications they are explicitly permitted to use.

Principle of Least Privilege

The principle of least privilege (PoLP) is a security concept in which a user, device, or process is granted only the minimum levels of access—or permissions—necessary to perform its function. In the context of network security, this means moving away from "fat" network access (where authenticated users can reach large segments of the network) to "thin," application-specific access. By enforcing least privilege through ZPA, organizations ensure that even if a user's credentials are compromised, the attacker cannot pivot to other systems because the network itself is invisible and inaccessible. This principle is foundational to reducing insider threats and limiting blast radius during a security incident.

Identity-Aware Access

Identity-aware access is a security model that bases access decisions on who the user is, rather than where they are connecting from. ZPA integrates directly with existing identity providers (IdPs) such as Microsoft Active Directory, Azure AD, Okta, or Ping Identity. This integration allows security teams to leverage existing user attributes, group memberships, and roles to build granular access policies. For example, a policy can state that only members of the "Finance" group, using a company-managed device, can access the "SAP Finance" application during business hours. This ensures that access is tightly coupled with the user's identity and context, reducing reliance on static IP addresses or network segments.

Application-Specific Access Policies

Application-specific access policies are the core enforcement mechanism within ZPA. Instead of creating rules that allow traffic to entire subnets or IP ranges, administrators define policies that target individual applications or groups of applications. These policies are written in plain language logic and can incorporate multiple conditions, including user identity, device posture, location, and time of day. ZPA allows for the segmentation of internal applications—such as HR systems, internal wikis, development servers, and database interfaces—so that each application has its own unique access policy. This micro-segmentation ensures that a user authenticated for one application has no

visibility or connectivity to any other application.

ZPA Architecture and Components

The ZPA architecture is composed of several key components that work together to provide secure, scalable access

1. Zscaler Cloud (Service Edge): A globally distributed cloud platform that acts as the broker between users and applications. It handles authentication, policy enforcement, and connection brokering without sitting in the data path.

2. App Connectors: Lightweight software VMs or containers deployed within the private data center or cloud environments. These connectors initiate outbound connections to the Zscaler cloud, eliminating the need for inbound firewall rules. They establish secure tunnels to the ZPA edge and forward traffic to the internal applications.

3. ZPA Client Connector: A software client installed on end-user devices (Windows, macOS, Linux, iOS, Android). It authenticates the user, checks device compliance, and establishes a micro-tunnel to the ZPA edge for application access.

4. Application Segments: Logical groupings of applications defined within the ZPA admin console. These segments represent the private resources that users need to access.

Implementation Methodology

The deployment of ZPA follows a structured, phased approach to ensure minimal disruption and maximum-security efficacy:

• Phase 1: Discovery and Inventory: Identify all internal applications currently in use. Categorize them by criticality, department, and user base. Tools like network scanners, configuration management databases (CMDB), and interviews with department heads are used to build a comprehensive application inventory.

• Phase 2: Connector Deployment: Deploy ZPA App Connectors in the data center or cloud environments where the applications reside. This involves provisioning virtual machines (using VMware, Hyper-V, or cloud-specific images) and configuring them to connect to the ZPA cloud.

• Phase 3: Application Onboarding: Define application segments within the ZPA console. This includes specifying the IP addresses and ports of the internal applications and associating them with the appropriate App Connector groups.

• Phase 4: Policy Creation: Integrate ZPA with the corporate identity provider (e.g., Azure AD). Create simulation mode before enforcement.

• Phase 5: Pilot and Rollout: Select a pilot group of users to test the new access method. Gather feedback, monitor performance, and refine policies. Gradually migrate user populations from legacy VPN access to ZPA.

• Phase 6: VPN Decommissioning: Once all users and applications are migrated, the legacy VPN infrastructure is decommissioned, eliminating a significant attack surface.

Tools and Technologies

• Primary Platform: Zscaler Private Access (ZPA) – The core Zero Trust access broker.

• Identity Provider (IdP): Microsoft Azure Active Directory (AD), Okta, or Ping Identity for user authentication and group synchronization.

• Endpoint Protection Platform (EPP): Integration with endpoint security tools (e.g., CrowdStrike, Sentinel One, Microsoft Defender for Endpoint) to assess device posture and hygiene before granting access.

• Security Information and Event Management (SIEM): Integration with SIEM platforms (e.g., IBM Q Radar, Splunk, LogRhythm) for centralized logging and correlation of access events.

• Virtualization Platforms: VMware vSphere, Microsoft Hyper-V, or public cloud environments (AWS, Azure, GCP) for hosting ZPA App Connectors.

• Infrastructure as Code (IaC): Tools like Terraform or Ansible for automated deployment and configuration of App Connectors at scale.

• Digital Experience Monitoring (DEM): Tools to monitor end-user connection quality and application performance.

Integration with Identity Providers

Integrating ZPA with the organization's existing identity provider is a critical step in establishing identity-aware access. This integration is typically done via SAML (Security Assertion Markup Language) or LDAP (Lightweight Directory Access Protocol). Once integrated, ZPA synchronizes user attributes and group memberships, allowing policies to reference these groups directly. For example, when a user authenticates via the ZPA Client Connector, they are redirected to the corporate IdP for

credentials and multi-factor authentication (MFA). Upon successful authentication, ZPA receives identity attributes that are then used to evaluate access policies and determine which applications the user is allowed to see and connect to.

Device Posture Checking

Beyond user identity, ZPA can enforce policies based on the security posture of the connecting device. Through integration with endpoint detection and response (EDR) and mobile device management (MDM) solutions, ZPA can assess factors such as:

- Antivirus software status and definition updates
- Disk encryption status (e.g., BitLocker, FileVault)
- Operating system patch level
- Presence of jailbreak or root compromise on mobile devices
- Real-time risk score from the EDR tool

If a device fails to meet the defined posture requirements (e.g., antivirus is disabled), ZPA can block access, grant limited access, or trigger a remediation workflow.

Micro-tunneling and Application Visibility

ZPA establishes a secure TLS/DTLS tunnel between the ZPA Client Connector and the ZPA service edge, and another tunnel between the service edge and the App Connector. Crucially, this creates a direct "app-to-user" micro-tunnel. The user's device is never placed on the corporate network; instead, traffic is proxied at Layer 7 (application layer). From the user's perspective, they can only resolve and connect to the specific DNS names or IP addresses of the applications they are authorized to access. All other internal resources remain completely invisible, preventing port scanning, vulnerability reconnaissance, and lateral movement.

Threat Intelligence Integration

To enhance security monitoring, ZPA can be integrated with threat intelligence platforms. Logs generated by ZPA—including connection attempts, policy violations, and user activity—can be forwarded to a SIEM (Security Information and Event Management) system. Within the SIEM, these logs can be correlated with threat intelligence feeds (such as those from MISP) to identify suspicious behavior. For example, if a user's credentials are used to access an application from an IP address associated with known malicious activity, the SIEM can trigger an alert, and integration with ZPA can automatically revoke that user's session.

Incident Response in a ZPA Environment

ZPA significantly streamlines the incident response process. When a potential threat is detected, security teams can leverage ZPA's capabilities to:

1. Isolation: Immediately revoke access for a compromised user account or device through the ZPA console, cutting off their connectivity to all internal applications.
2. Forensics: Access detailed logs within ZPA showing exactly which applications were accessed, by whom, from which location, and for how long.
3. Containment: Because users have no network-level access, a compromised endpoint cannot be used to scan or attack other internal systems, containing the blast radius.

4. Granular Blocking: Temporarily block access to specific high-risk applications while allowing access to others, ensuring business continuity during an investigation.

Understanding ZPA: Key Takeaways

Zscaler Private Access represents a fundamental shift in how remote access is architected and secured. Here are the key aspects to understand about the solution:

1. It is cloud-native: ZPA is delivered entirely from the Zscaler cloud, eliminating the need for hardware appliances and complex VPN head-end management.

2. It is identity-centric: Policies are built around users and groups, not IP addresses, making access control more intuitive and aligned with organizational structure.

3. It ensures network invisibility: By never placing users on the network, ZPA renders internal infrastructure invisible to the internet, preventing reconnaissance and DDoS attacks.

4. It is scalable: The cloud architecture allows ZPA to scale instantly to accommodate thousands of concurrent users without performance degradation.

5. It supports hybrid environments: ZPA works seamlessly for applications hosted in on-premises data centers, private clouds, and public cloud environments.

6. It improves user experience: Users get direct, optimized connections to applications without back-hauling traffic through a centralized VPN concentrator, reducing latency.

Conclusion

Stage 1: Web Application Testing

Web application testing is an essential process that ensures the overall quality, security, functionality, and reliability of web-based applications. In today's digital environment, organizations heavily depend on web applications to deliver services, manage operations, and communicate with users. Therefore, thorough testing becomes necessary to identify defects, security vulnerabilities, and performance issues before the application is released to the public. The testing process involves evaluating multiple aspects such as user interface design, application functionality, compatibility across browsers and devices, application performance under heavy traffic, and security against

potential cyber threats. By performing these tests, developers and security professionals can detect problems early in the development lifecycle and fix them before they impact end users. This improves the overall user experience and enhances the trustworthiness of the application.

Key outcomes of web application testing include:

- Ensuring that the application works correctly and meets user requirements.
- Detecting bugs, vulnerabilities, and errors before deployment.
- Improving application performance and stability.
- Ensuring compatibility across multiple browsers, operating systems, and devices.
- Strengthening the security of the application by identifying potential threats.
- Delivering a seamless and reliable user experience.

Overall, effective web application testing contributes to delivering high-quality applications while protecting both organizations and users from potential risks and failures.

Stage 2: Vulnerability Assessment using Nessus

The vulnerability assessment performed using Nessus provided a detailed analysis of the website's security posture. The scan identified several vulnerabilities present within the system and classified them based on their severity levels such as critical, high, medium, and low. This classification helps security teams prioritize which vulnerabilities need immediate attention and which ones can be addressed later. The report provided detailed descriptions of each vulnerability along with possible impacts and recommended remediation strategies.

The assessment also highlighted common security issues such as outdated software components, missing security patches, configuration weaknesses, and potential exposure of sensitive information. Addressing these vulnerabilities is crucial to preventing unauthorized access, data breaches, and service disruptions. Additionally, the findings were aligned with industry-recognized security standards such as the Open Worldwide Application Security Project guidelines, ensuring that the application follows best security practices.

Important insights obtained from the vulnerability assessment include:

▪ Identification of critical and high-risk vulnerabilities that require immediate remediation.

▪ Detection of outdated software components and missing security patches.

▪ Identification of configuration errors that may expose the system to cyber attacks.

▪ Recommendations for patching, reconfiguration, and security improvements.

▪ Alignment with industry security standards and best practices.

▪ Emphasis on continuous monitoring and regular vulnerability scanning.

Overall, the Nessus vulnerability report provided a clear understanding of the system's security status and emphasized the importance of proactive security management to protect applications from evolving cyber threats.

Stage 3: Implementation of Zero Trust using Zscaler Private Access

The implementation of Zscaler Private Access (ZPA) represents a major advancement in modern cybersecurity architecture. Traditional security models relied on network-based trust where users inside the corporate network were automatically trusted. However, this approach is no longer sufficient in modern environments where employees work remotely and applications are hosted across multiple cloud platforms. To address these challenges, organizations are adopting the Zero Trust Architecture, which follows the principle of "never trust, always verify."

ZPA enables secure access to private applications without exposing them directly to the internet. Instead of providing full network access like traditional VPNs, ZPA connects authenticated users directly to specific applications through secure micro-tunnels. This approach reduces the attack surface and prevents attackers from moving laterally across the network if a system is compromised. Additionally, ZPA enforces identity-based access control policies that ensure users can only access applications necessary for their roles.

Benefits of implementing ZPA include:

▪ Enforcing least-privilege access policies based on user identity.

▪ Making private applications invisible to the public internet.

▪ Preventing lateral movement and network-based attacks.

▪ Providing secure and fast application access for remote employees.

▪ Reducing dependency on traditional VPN infrastructure.

▪ Enhancing visibility and monitoring of user-to-application interactions.

Ultimately, the adoption of Zero Trust security through ZPA provides organizations with a modern, scalable, and secure framework that supports digital transformation while protecting critical resources from cyber threats.

Future Scope

Stage 1: Future Scope of Web Application Testing

The future scope of web application testing is expanding rapidly as digital technologies continue to evolve. With the growing number of online services, web applications are becoming increasingly complex and interconnected. This creates a greater need for advanced testing methodologies to ensure application reliability, performance, and security. Organizations are investing heavily in automated testing tools and security testing frameworks to detect vulnerabilities and improve software quality throughout the development lifecycle.

Some important future developments in web application testing include:

▪ Increased Demand: As businesses continue moving their services online, the demand for secure and reliable web applications will grow, increasing the need for comprehensive testing processes.

▪ Advanced Technologies: Emerging technologies such as Artificial Intelligence, Internet of Things, and blockchain will introduce new testing challenges and require specialized testing tools.

▪ Security-Focused Testing: With the increasing number of cyber attacks and data breaches, security testing will become a critical component of the development lifecycle.

Stage 2: Future Scope of Vulnerability Testing

The future scope of vulnerability testing is becoming increasingly important due to the rapid expansion of digital infrastructure and the growing sophistication of cyber threats. Organizations

must continuously monitor their systems for vulnerabilities and implement proactive security measures to prevent potential attacks.

Future developments in vulnerability testing include:

▪ AI-Driven Testing: Artificial intelligence and machine learning will automate vulnerability detection by analyzing large volumes of data and identifying hidden security risks.

▪ IoT Security Testing: As the number of Internet of Things devices increases, security testing will extend to ensure these devices and their communication networks remain secure.

▪ Blockchain Security: Testing will focus on protecting blockchain applications, smart contracts, and cryptocurrency transactions from security vulnerabilities.

Stage 3: Future Scope of Zero Trust Security

The successful deployment of Zero Trust security using Zscaler Private Access opens opportunities for further enhancements and integration with other advanced security technologies. Organizations can expand their Zero Trust frameworks to create a more comprehensive and intelligent security ecosystem.

Future improvements in Zero Trust implementation include:

▪ Integration with Zscaler Internet Access: Combining ZPA with Zscaler Internet Access to create a unified Secure Access Service Edge (SASE) platform for managing both internet and private traffic.

▪ Advanced Device Posture Checks: Implementing real-time risk assessments based on device health and security status before granting access.

▪ Privileged Remote Access: Providing secure, time-limited access for third-party vendors and contractors to specific systems.

▪ IoT and OT Security: Applying Zero Trust principles to Internet of Things and operational technology environments to prevent unauthorized device communication.

▪ AI-Based Threat Detection: Using machine learning algorithms to analyze access logs and detect unusual user behavior or suspicious activity.

▪ Cloud Workload Protection: Extending Zero Trust policies to secure communications between

cloud-based applications and services across multi-cloud environments.

Sample code

```
{
  "policyName": "Finance_App_Access",
  "policyType": "ACCESS_POLICY",
  "description": "Allow only finance team to access finance application",
  "conditions": {
    "userGroups": [
      "Finance_Team"
    ],
    "devicePosture": [
      "Company_Laptop"
    ],
    "location": [
      "India_Office"
    ]
  },
  "applications": [
    "finance.internal.company.com"
  ],
  "action": "ALLOW"
}
{
  "policyName": "Block_External_Access",
  "policyType": "ACCESS_POLICY",
  "description": "Block access from unmanaged devices",
```

```
  "conditions": {
    "devicePosture": [
      "Unmanaged_Device"
    ]
  },
  "applications": [
    "internal.company.com"
  ],
  "action": "DENY"
}
# Vulnerable Code Example

from flask import Flask, request

app = Flask(__name__)

@app.route('/view')
def view_file():
    filename = request.args.get("file")
    with open("files/" + filename, "r") as f:
        content = f.read()
    return content

app.run()
import os
from flask import Flask, request
```

```
app = Flask(__name__)

BASE_DIR = "files"

@app.route('/view')
def view_file():
    filename = request.args.get("file")
    filepath = os.path.join(BASE_DIR, filename)

    if not filepath.startswith(BASE_DIR):
        return "Access Denied"

    with open(filepath, "r") as f:
        return f.read()

app.run()
# Vulnerable Code Example

import base64

password = "admin123"

encoded = base64.b64encode(password.encode())

print(encoded)
```

```
# Secure Password Hashing

import bcrypt

password = b"admin123"

hashed = bcrypt.hashpw(password, bcrypt.gensalt())

print(hashed)
# Vulnerable Code

username = input("Enter username:")
password = input("Enter password:")

query = "SELECT * FROM users WHERE username='" + username + "' AND password='" +
password + "'"
users = {
    "admin": "admin123",
    "user": "user123"
}

username = input("Enter username: ")
password = input("Enter password: ")

if username in users and users[username] == password:
```

```
    print("Login successful")
else:
    print("Invalid credentials")
import logging


logging.basicConfig(filename="security.log",
                    level=logging.INFO)


username = input("Username: ")


logging.info("Login attempt by: " + username)


print("Attempt logged")
try:
    number = int(input("Enter number: "))
    result = 100 / number
    print(result)


except ZeroDivisionError:
    print("Cannot divide by zero")


except ValueError:
    print("Invalid input")
from flask import Flask, request
import os
```

```
app = Flask(__name__)


UPLOAD_FOLDER = "uploads"


@app.route("/upload", methods=["POST"])
def upload_file():
    file = request.files["file"]
    filename = file.filename
    path = os.path.join(UPLOAD_FOLDER, filename)
    file.save(path)
    return "File uploaded successfully"


app.run()
def validate_input(user_input):
    if user_input.isalnum():
        print("Valid Input")
    else:
        print("Invalid Input")


data = input("Enter username: ")
validate_input(data)
import os


AUTHORIZED_FOLDER = "secure_files"


filename = input("Enter file name: ")
```

```
filepath = os.path.join(AUTHORIZED_FOLDER, filename)

if os.path.exists(filepath):
    with open(filepath, "r") as file:
        print(file.read())
else:
    print("File not found or access denied")
import random

otp = random.randint(100000,999999)
print("OTP sent to user:", otp)

user_otp = int(input("Enter OTP: "))

if user_otp == otp:
    print("Authentication Successful")
else:
    print("Authentication Failed")
import uuid

sessions = {} import datetime

def log_activity(activity):
    time = datetime.datetime.now()
    log = open("activity_log.txt","a")
```

```
    log.write(str(time) + " - " + activity + "\n")
    log.close()

log_activity("User login attempt")
log_activity("File access request")
print("Activities logged successfully")
def create_session(user):
    session_id = str(uuid.uuid4())
    sessions[session_id] = user
    return session_id

user = "admin"
session = create_session(user)

print("Session created:", session)
import re

def check_password_strength(password):
    if len(password) < 8:
        print("Password too short")
    elif not re.search("[A-Z]", password):
        print("Password must contain uppercase letter")
    elif not re.search("[0-9]", password):
        print("Password must contain number")
    elif not re.search("[@#$%^&*]", password):
        print("Password must contain special character")
```

```
    else:
        print("Strong Password")

pwd = input("Enter password: ")
check_password_strength(pwd)
from cryptography.fernet import Fernet

key = Fernet.generate_key()
cipher = Fernet(key)

data = input("Enter data to encrypt: ")

encrypted = cipher.encrypt(data.encode())
print("Encrypted Data:", encrypted)

decrypted = cipher.decrypt(encrypted).decode()
print("Decrypted Data:", decrypted)
```